

Ejercicio 04: FastAPI + Database

Objetivo

Integrar SQLAlchemy con FastAPI usando Dependency Injection para crear una API CRUD completa.

Duración estimada: 45 minutos

Pasos

Paso 1: Estructura de Archivos

El ejercicio usa una estructura simple:

```
starter/  
├─ main.py      # Todo en un archivo para simplicidad  
└─ notes.db     # SQLite (se crea automáticamente)
```

Abre `starter/main.py` para comenzar.

Paso 2: Configuración de Base de Datos

Setup de Engine, Base y SessionLocal:

```
engine = create_engine("sqlite:///./notes.db")  
SessionLocal = sessionmaker(bind=engine)  
  
class Base(DeclarativeBase):  
    pass
```

Descomenta la sección del Paso 2.

Paso 3: Modelo SQLAlchemy

Modelo `Note` para la base de datos:

```
class Note(Base):  
    __tablename__ = "notes"  
  
    id: Mapped[int] = mapped_column(primary_key=True)  
    title: Mapped[str] = mapped_column(String(100))  
    content: Mapped[str] = mapped_column(Text)  
    created_at: Mapped[datetime] = mapped_column(default=datetime.utcnow)
```

Descomenta la sección del Paso 3.

Paso 4: Schemas Pydantic

Schemas para validación de request/response:

```
class NoteCreate(BaseModel):  
    title: str = Field(..., min_length=1, max_length=100)  
    content: str
```

```
class NoteResponse(BaseModel):
    id: int
    title: str
    content: str
    created_at: datetime

    model_config = ConfigDict(from_attributes=True)
```

- `NoteCreate` : Valida datos de entrada
- `NoteResponse` : Serializa datos de salida
- `from_attributes=True` : Permite crear desde objeto SQLAlchemy

Descomenta la sección del Paso 4.

Paso 5: Dependency `get_db`

La función que provee la sesión:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- `yield` : Permite que FastAPI use la sesión
- `finally` : Garantiza que se cierre aunque haya error

Descomenta la sección del Paso 5.

Paso 6: Endpoint `CREATE`

```
@app.post("/notes", response_model=NoteResponse, status_code=201)
def create_note(note: NoteCreate, db: Session = Depends(get_db)):
    db_note = Note(title=note.title, content=note.content)
    db.add(db_note)
    db.commit()
    db.refresh(db_note)
    return db_note
```

Descomenta la sección del Paso 6.

Paso 7: Endpoints `READ`

```
@app.get("/notes", response_model=list[NoteResponse])
def list_notes(db: Session = Depends(get_db)):
    stmt = select(Note).order_by(desc(Note.created_at))
    return db.execute(stmt).scalars().all()

@app.get("/notes/{note_id}", response_model=NoteResponse)
def get_note(note_id: int, db: Session = Depends(get_db)):
    note = db.get(Note, note_id)
    if not note:
```

```
        raise HTTPException(status_code=404, detail="Note not found")
    return note
```

Descomenta la sección del Paso 7.

Paso 8: Endpoints UPDATE y DELETE

```
@app.put("/notes/{note_id}", response_model=NoteResponse)
def update_note(note_id: int, note_data: NoteUpdate, db: Session = Depends(get_db)):
    note = db.get(Note, note_id)
    if not note:
        raise HTTPException(status_code=404, detail="Note not found")
    # Actualizar campos
    for key, value in note_data.model_dump(exclude_unset=True).items():
        setattr(note, key, value)
    db.commit()
    return note

@app.delete("/notes/{note_id}", status_code=204)
def delete_note(note_id: int, db: Session = Depends(get_db)):
    note = db.get(Note, note_id)
    if not note:
        raise HTTPException(status_code=404, detail="Note not found")
    db.delete(note)
    db.commit()
```

Descomenta la sección del Paso 8.

Ejecutar

```
# Iniciar servidor
uv run fastapi dev starter/main.py

# O con uvicorn directamente
uv run uvicorn starter.main:app --reload
```

Abre <http://localhost:8000/docs> para ver Swagger UI.

Probar la API

Crear nota

```
curl -X POST http://localhost:8000/notes \
-H "Content-Type: application/json" \
-d '{"title": "Mi primera nota", "content": "Contenido de prueba"}'
```

Listar notas

```
curl http://localhost:8000/notes
```

Obtener una nota

```
curl http://localhost:8000/notes/1
```

Actualizar nota

```
curl -X PUT http://localhost:8000/notes/1 \
-H "Content-Type: application/json" \
-d '{"title": "Título actualizado"}'
```

Eliminar nota

```
curl -X DELETE http://localhost:8000/notes/1
```

✓ Verificación

- ☐ El servidor inicia sin errores
- ☐ Swagger UI muestra los 5 endpoints
- ☐ POST /notes crea una nota y retorna con ID
- ☐ GET /notes lista todas las notas
- ☐ GET /notes/{id} retorna una nota o 404
- ☐ PUT /notes/{id} actualiza campos
- ☐ DELETE /notes/{id} elimina la nota

🧠 Flujo de Datos

```
Request → Pydantic (valida) → Endpoint → SQLAlchemy → DB
                                         ↓
Response ← Pydantic (serializa) ← Endpoint ← SQLAlchemy
```

[← Anterior: CRUD](#) | [Volver a Prácticas →](#)